



21-DEFINITION-OF-DONE.md — The phase exit checklist Claude Code cannot fake



A coding agent will always tell you a phase is complete. This file replaces its judgement with a checklist that requires **command output as evidence** for every claim. No evidence, no PASS.

1. The evidence rule

Every item below is marked PASS only with a pasted command and its real output. The following are **not** evidence and are automatic FAIL:

- "Implemented as specified."
- "Tests should pass."
- "This follows the contract."
- A test file that exists but was never run.
- A test that passes because it asserts nothing, or because the assertion was weakened to fit the code.

2. Universal Definition of Done — every phase, no exceptions

Contracts

- ☐ Every type, table, schema and IPC message this phase introduces is placed in a frozen-contract location and added to `contracts.lock`.

☐ No existing frozen contract was modified, or an ADR is attached with a grep-based blast radius.

☐ `cargo xtask contracts --check` output pasted, clean.

Errors

☐ Every failure path returns a coded `AuraError` with severity, recovery, context and a user message.

☐ New codes are registered in `errors.toml` within the correct domain range.

☐ Banned-pattern gate output pasted, clean.

☐ Every new code is reachable and has a runbook file.

Determinism

☐ Repeat test: same input, 5 runs, identical digests. Output pasted.

☐ Thread-count sweep (1, 8, 32) identical. Output pasted.

☐ No new `HashMap` iteration, unseeded RNG, wall-clock read, or unordered float reduction in any output-affecting path.

Concurrency and resume

☐ Every new task type is idempotent; double-run snapshot test pasted.

☐ Kill-and-resume test at 3 points passes with identical results.

☐ No unbounded channels; no second SQLite writer; no lock held across `.await`.

Resources

☐ New budgets added to `perf/budgets.toml` with target and ceiling.

☐ Measured values pasted, all within ceiling on T-Typ.

☐ Phase completes correctly on T-Min, even if slower.

☐ Leak check: RSS returns to baseline within 5 %.

Quality

☐ Unit tests for every public function; coverage on new code at least 80 %.

☐ At least one test per acceptance criterion in the phase spec, named after it.

- ☐ For every guard added, a test proving the guard fires when the guard is removed.
- ☐ Golden corpus unchanged, or regenerated in a separate commit with sign-off.

Observability

- ☐ New stages emit structured logs with the mandatory field set.
- ☐ New decisions write decision-ledger rows and are reversible.
- ☐ New metrics registered; no high-cardinality labels.
- ☐ Redaction test still green with any new sink or field.

Security and ethics

- ☐ No new network call outside the governed gateway.
- ☐ No client imagery leaves the machine without the consent flag.
- ☐ Nothing in this phase can change what a person looks like.
- ☐ Any new `unsafe` is inside a `-sys` crate with a `SAFETY:` comment.

Product

- ☐ Every user-visible failure has a plain-English message with a next action.
- ☐ The feature works fully offline.
- ☐ Progress and cancellation behave correctly for any long operation.
- ☐ Docs and in-app help updated by `DOC`.

Review

- ☐ Implemented under one role hat, reviewed under a different one, with the review findings recorded.
- ☐ `EM` phase exit report written: what shipped, what was deferred, what surprised us.
- ☐ `CTO` sign-off on contracts and invariants.

3. Phase-type specific additions

Phase type	Extra required evidence
Model phase (05–12, 15–22, 24)	All five model artefacts; <code>model-gate</code> output with per-slice table; ECE; cross-EP parity; blind A/B if it changes pixels
GPU phase (14, 18, 19, 22)	CPU reference implementation; parity test; device-lost fallback test; VRAM squeeze at 2 GB; odd-dimension test
Colour phase (15, 16, 25, 26)	ΔE_{2000} report on the colour corpus; skin-tone protection test; <code>COL</code> sign-off; gallery-consistency variance floor check
Cloud phase (04, 07, 13, 27)	Schema validation tests; prompt-injection test with hostile EXIF; cost-cap test; offline fallback test; response-cache determinism test
IO / catalog phase (01, 02, 30)	Migration chain; corrupt-input corpus; disconnect test; long-path and Unicode fixtures; dual-instance lock test
Orchestration phase (28)	Full kill -9 ladder at five points; 8-hour soak; cancellation storm; honest-progress verification

4. The phase exit report template

```
# PHASE-14 EXIT REPORT

## Status: COMPLETE | COMPLETE WITH DEBT | NOT COMPLETE

## Evidence table
| DoD item | Command | Result |
|---|---|---|
| Repeat determinism | `cargo test -p aura-render determinism::repeat` | 5/5 identical, digest 8f2a... |
| Budget: develop p95 | `cargo xtask bench develop` | 11m42s vs target 12m, ceiling 18m |

## Acceptance criteria
| # | Criterion | Test name | Result |

## Contracts introduced
## ADRs written
## Known debt (with owner and due date)
```

```
## Surprises and what we learned
## Next phase readiness: dependencies met? y/n
```

"Complete with debt" is allowed exactly twice per epic, and every debt item needs an owner and a date. A third forces a stop-and-fix week. This is the mechanism that prevents thirty phases of accumulating shortcuts.

5. The reviewer's adversarial pass

After implementation, the agent must switch hats and genuinely attack the work:

1. *"Which acceptance criterion is only superficially satisfied?"*
2. *"Which test would still pass if I deleted the feature?"* — delete it and show the test failing.
3. *"What input would break this? A 0-byte file, a 61 MP file, a file with 400 faces, a wedding at midnight in candlelight."*
4. *"What happens if this runs twice? If it is killed halfway? If the disk fills at exactly this point?"*
5. *"Which error path has never been executed by any test?"*
6. *"Does anything here make the output different on a different machine?"*
7. *"Would a photographer understand what happened if this failed?"*

The answers go into the exit report. A review that finds nothing is itself a finding — and is rejected.

6. Anti-patterns that void a PASS

- Tests written after the fact to match whatever the code happens to do.
- Assertions loosened rather than bugs fixed.
- `#[ignore]` added to a failing test.
- A tolerance widened without an ADR.
- Golden manifest regenerated in the same commit as a code change.
- A guard removed "temporarily".

- A budget ceiling raised to make the benchmark pass.
- Work marked done while the feature only works on the developer's machine.

7. Claude Code prompt

Act as `QAL` (QA Lead — Automation). The implementation of PHASE-NN claims to be complete. Walk every item of the universal Definition of Done in `21-DEFINITION-OF-DONE.md`, plus the phase-type additions that apply. For each item, run the command and paste the real output. Mark PASS or FAIL — never "probably". Then perform the adversarial pass in section 5 and answer all seven questions concretely, including actually deleting a feature to show its test fails. Finally, produce the exit report using the template. If any item fails, say NOT COMPLETE and list exactly what must be fixed. Do not fix anything in this pass; audit only.